

Blink Match Memory

A Machine Problem Proposal by

Daclizon, Angel B. - 2025-15978

Lasaca, April Marie S. - 2025-09838

Magallanes, Roswell Benedict T. - 2025-07851

Tatierra, Clarenz Ken M. - 2025-11343

In Partial Fulfillment of the Requirements

for the course Math 154

University of the Philippines

TACLOBAN COLLEGE

Tacloban City

May 14, 2026

I. Game Introduction

Blink Match Memory is an interactive card-matching game which will be developed using Java with a graphical user interface(GUI). It is inspired by the traditional memory-matching game wherein the users flip the cards to find identical pairs and recall each card they flipped in order for them to successfully match all the cards.

The proposed game introduces a version which gives the players a fast-paced experience as it contains a time pressure system. The players must quickly recall the positions of each card and match pairs before the time runs out. Also, the players must easily adapt to the unexpected disruptions in the gameplay as there will be a random shuffling of the cards during the game.

Another proposed feature for Blink Match memory is the incorporation of a weather system in which there will be an environmental change which signals that the cards will shuffle.

Objectives

- The main objective of the game is to match all of the card pairs on or before the time runs out and adapt to the unexpected disruptions while the player achieves the highest possible score.
- Implement a robust OOP Hierarchy where cards, the game engine, and the weather system are treated as distinct objects with encapsulated data.
- Use Multi-threading to manage the countdown timer and the "Stormy Weather" shuffle logic simultaneously without interrupting the main GUI thread.
- Ensure Data Persistence by implementing a system to save and load high scores

II. Controls and Mechanics

The game consists of a grid of face-down cards arranged in pairs. Then the player will select two cards per turn. If the selected pairs match, the cards remain face-up and the player will earn points. On the other hand, if the selected cards do not match, the cards will flip back after a short delay.

Moreover, if the weather shifts from sunny to stormy, it will serve as a signal that the cards will shuffle so the player should be able to adapt to these changes and pair the cards before the time runs out. The game will continue until all cards are matched or the timer reaches zero.

In terms of the time pressure system, the game will include a countdown timer, so the player must complete all matches before time runs out.

While for the weather-based disruption, if the weather is sunny, then the player experiences normal gameplay. However, if it shifts to stormy weather, the player will be in a disruption phase as the cards will be shuffled, making it harder to match the cards.

Controls (Desktop):

Mouse Controls

- Left Click - flip or selects a card
- press Restart and Exit buttons.

Keyboard Controls

S key - Start game

R key - Restart game

E key - Exit game

Obstacles include:

- Time Constraint
- Storm Disruption

The game ends when the time runs out or when you finish matching all cards before the time runs out.

Technical Methodology

System Architecture Overview

- This project follows a Model-View-Controller (MVC) architecture to ensure maintainability and scalability.

1. Model (Game Data Layer)

Handles all game data and logic:

A. Card - it represents individual cards.

B. Player- stores score and player data.

C. GameState- its role is to track time, moves and number of matches.

D. Weather (Abstract Class)

- SunnyWeather

-StormyWeather

2. View (User Interface Layer)

Handles all graphical components:

GameWindow - main JFrame

GamePanel - gameplay interface

StartPanel - start menu

HelpPanel - instructions

GameOverPanel - end screen

3. Controller (Logic Layer)

Controls game flow and interactions:

Game Panel(this acts as controller in our design)

Handles:

- Card Selection
- Matching logic
- Weather triggers
- Timer updates

❖ Core Gameplay Classes

Card

- **Attributes:**
 - id, symbol, isFlipped, isMatched
- **Methods:**
 - flip()
 - checkMatch(Card other)

GameState

- **Tracks:**
 - Remaining time
 - Moves made
 - Matched pairs
- **Methods:**
 - incrementMoves()
 - updateTimer()
 - checkWinCondition()

Player

- **Attributes:**
 - name, score
- **Methods:**
 - addScore()
 - reset()

Weather System (Key Innovation – emphasize this in defense)

Abstract Class:

Weather

- **Methods:**
 - applyEffect()
 - getDescription()

SunnyWeather

- Normal gameplay
- No disruption

StormyWeather

- **Triggers:**
 - Card shuffle
 - Visual effects

❖ Game Flow Components

GamePanel (Main Logic Hub)

Handles:

- Card flipping
- Matching validation
- Timer updates
- Weather switching
- UI updates

StartPanel

- Start button
- Initializes game

HelpPanel

- Displays instructions

GameOverPanel

- **Displays:**
 - Score
 - Restart option

Additional System Components

SoundPlayer

- **Plays:**
 - Card flip sound
 - Match success
 - Storm sound

BasePanel (Reusable UI Class)

- Common layout and styling
- Reduces code duplication

❖ Multithreading Implementation

The system uses Java Timer / Threading to:

- Run countdown timer independently
- Prevent UI freezing
- Handle:
 - Delays (card flip back)

- Weather-triggered shuffle timing

Core Logic

- Card matching algorithm
- Shuffle algorithm during storm
- Score calculation system
- Timer synchronization

Core Gameplay Features

- Card flipping system
- Matching validation
- Score tracking
- Win/Lose conditions

The Core Loop

1. Selection

The player flips two face-down cards.

2. Matching:

- **Match-** The cards stay face-up, then points are awarded.
- **Mismatch-** The cards flip back after a brief delay (using a Timer thread).

3. The Storm System:

- **Sunny Weather-** Standard gameplay.
- **Stormy Weather-** Triggered randomly or at intervals. A visual shift (background change) warns the player before the remaining face-down cards are shuffled, forcing a re-evaluation of card positions.

4. Win/Loss Condition

The game ends when all pairs are found (Win) or the timer hits zero (Loss).

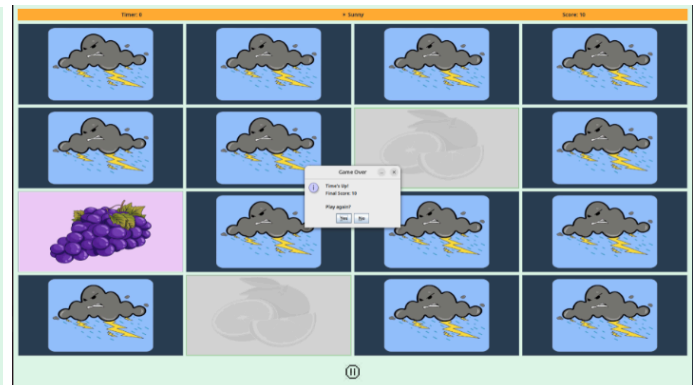
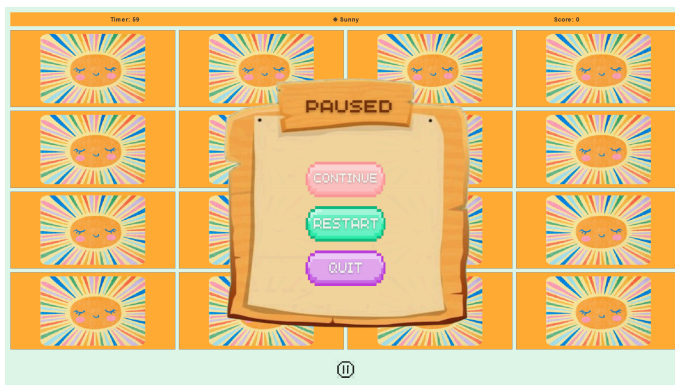
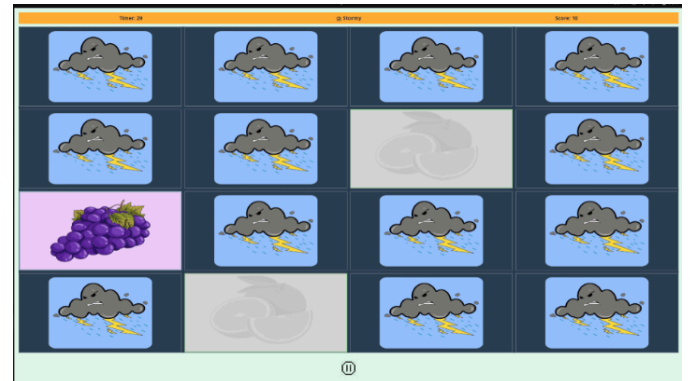
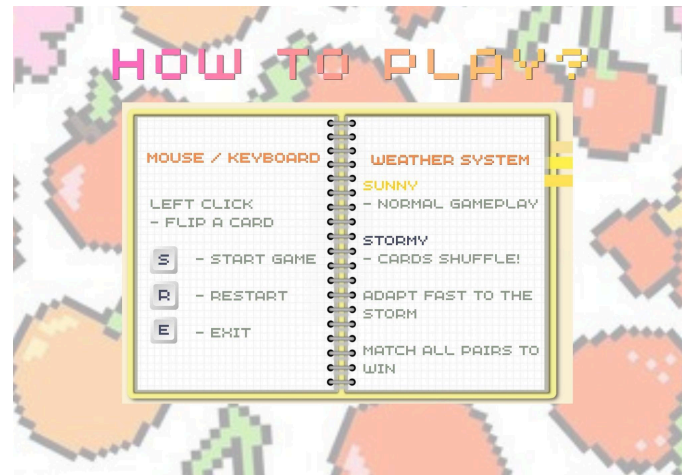
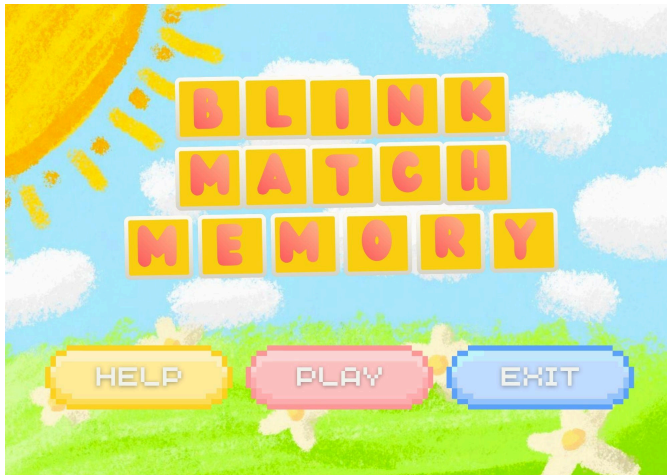
III. Existing UI Design (if game is existing)



<https://www.alamy.com/memory-game-smartphone-interface-vector-template-mobile-app-page-gradient-design-layout-match-pair-concentration-intellectual-game-screen-flat-ui-image339261295.html>

The existing memory match game UI design contains titles and instructions, card grid layout, a progress bar and a dark themed background design. The game displays “Memory Match” at the top and has an instruction below it. Moreover, the game board has a 4x3 grid of cards which are initially in face down position to encourage memory recall. Furthermore, a progress bar is visible under the instructions, which indicates the number of attempts used out of the total attempts for the game. Lastly, the dark themed background design enhances the readability and the visual appeal of the game, which helps the players to keep their focus on the game and avoids visual clutter.

IV. Proposed UI Design



V. Table of Activities

Objective	W1	W2	W3	W4	W5	W6	W7	W8
1. Planning and Concept (game idea, features, research)								
2. UI Design								
3. Coding Setup (Java project, classes, basic structure)								
4. Coding Player Mechanics								
5. Coding Platform System								
6. Coding Power-ups & Obstacles								
7. GUI Implementation								
8. Game Logic & Scoring System								
9. Debugging & Testing (fixing errors, improving gameplay)								
10. Finalization & Presentation (polish, documentation, demo)								